

## ĐỒ ÁN TỐT NGHIỆP

# Hệ thống giao việc và chia sẻ tài liệu số trong tổ chức

Kiến trúc Microservices - Node.js / NestJS - TypeScript

Case study: Ủy ban nhân dân

Ngành: Công nghệ phần mềm - Lớp D22CQCNP01-N

### Nhóm C17

- N22DCCN068 - Nguyễn Lưu Tấn Sang
- N22DCCN071 - Vũ Ngọc Sơn
- N22DCCN088 - Đỗ Xuân Trí

Tài liệu: Kịch bản thuyết trình + lý thuyết + kiến trúc + luồng hệ thống + bảo mật + ghi log

Thời lượng trình bày: 20 phút

Ghi chú nguyên tắc trình bày: chỉ khẳng định phần đã kiểm tra thực tế trên server; không đưa password, JWT, secret, object key hoặc thông tin database lên màn hình.

## 1. Giới thiệu chung

### 1.1. Đề tài

Đề tài: **Xây dựng ứng dụng giao việc và chia sẻ tài liệu số trong tổ chức**, định hướng áp dụng cho môi trường cơ quan nhà nước (case study: Ủy ban nhân dân). Hệ thống giải quyết hai nghiệp vụ chính: (1) giao việc và theo dõi công việc; (2) chia sẻ tài liệu theo đúng người, đúng quyền, đúng thời hạn, có ghi nhận vết đầy đủ.

## 1.2. Bài toán nghiệp vụ tại UBND

Trong một UBND, một công việc thường liên quan đến nhiều phòng ban và nhiều cán bộ. Kèm theo mỗi công việc là hồ sơ, văn bản, dự thảo - vốn là tài liệu nhạy cảm. Các vấn đề thực tế:

- Công việc phân tán qua email, giấy tờ, thiếu kênh thống nhất để giao và theo dõi.
- Không kiểm soát được ai được xem, ai được tải, ai được chia sẻ tiếp; quyền tồn tại vô thời hạn.
- Không có dấu vết ai đã truy cập tài liệu nào, khi nào - khó đối chiếu, khó điều tra sự cố.
- Sau khi công việc kết thúc, quyền truy cập vẫn có thể còn hiệu lực - rủi ro rò rỉ dữ liệu.

## 1.3. Mục tiêu và giải pháp

- Giao việc theo vòng đời: task có creator, assignee, deadline, trạng thái, task con, comment, nộp kết quả, phê duyệt.
- Quyền tài liệu gắn với công việc (task-derived grant): không có task làm căn cứ thì không có quyền; quyền tự hết hạn theo deadline.
- Bảo mật nội dung: quét virus, mã hóa AES-256-GCM, chữ ký toàn vẹn, tải xuống qua ticket dùng một lần.
- Ghi vết (audit log) chống giả mạo và giám sát bảo mật theo ngưỡng cảnh báo.

## 1.4. Đối tượng sử dụng

- **Người giao việc** (lãnh đạo/phòng ban): tạo task, giao việc, chia sẻ tài liệu, giới hạn quyền, phê duyệt kết quả.
- **Người được giao** (chuyên viên): xem task/tài liệu được chia sẻ, cập nhật tiến độ, nộp kết quả; có thể giao tiếp việc nhưng không vượt quyền gốc.
- **Admin**: quản lý người dùng, vai trò, capability, khóa tài khoản, cấu hình cảnh báo. Theo nguyên tắc tách biệt trách nhiệm, Admin không mặc định xem/chia sẻ nội dung tài liệu.

# 2. Case study: Ủy ban nhân dân

## 2.1. Bối cảnh

Lấy bối cảnh một UBND cấp huyện. Lãnh đạo UBND giao nhiệm vụ cho các phòng chuyên môn; mỗi nhiệm vụ kèm hồ sơ, văn bản cần xử lý. Cán bộ xử lý xong phải nộp kết quả để lãnh đạo duyệt. Toàn bộ quá trình phải kiểm soát được ai tiếp cận tài liệu nào và trong bao lâu.

## 2.2. Các vai trò trong case study

| Vai trò                             | Mô tả                                                                    |
|-------------------------------------|--------------------------------------------------------------------------|
| Lãnh đạo UBND (người giao việc)     | Tạo công việc, giao cho phòng chuyên môn, chia sẻ hồ sơ, duyệt kết quả.  |
| Chuyên viên phòng (người được giao) | Nhận việc, xem/tải tài liệu được chia sẻ, xử lý và nộp kết quả.          |
| Cán bộ phụ trách lưu trữ            | Nhận tài liệu sau khi công việc hoàn thành (capability ARCHIVE_RECEIVE). |
| Admin hệ thống                      | Quản lý tài khoản, vai trò, khóa/mở khóa, cấu hình cảnh báo bảo mật.     |

## 2.3. Ví dụ một chu kỳ công việc cụ thể

1. Lãnh đạo đăng nhập, tạo task "Xử lý hồ sơ đề xuất dự án chuyển đổi số", giao cho chuyên viên, đặt deadline.
2. Lãnh đạo upload hồ sơ (DOCX/PDF), gắn vào task, cấp quyền **PREVIEW**, **DOWNLOAD** có thời hạn cho chuyên viên.
3. Chuyên viên đăng nhập: chỉ thấy đúng task và tài liệu được phép; xem/tải qua ticket bảo mật.
4. Chuyên viên xử lý, nộp kết quả; task chuyển sang trạng thái chờ review.
5. Lãnh đạo review và phê duyệt; task chuyển sang APPROVED.
6. Quyền tài liệu hết hạn theo deadline; mọi thao tác đều được ghi vào audit log.

## 2.4. Giá trị mang lại

- Một kênh thống nhất để giao việc và theo dõi tiến độ giữa các phòng ban.
- Quyền truy cập được cấp tối thiểu, có thời hạn, tự động hết hiệu lực khi công việc kết thúc.
- Mọi truy cập đều có dấu vết; hệ thống cảnh báo khi phát hiện hành vi bất thường.

## 3. Kịch bản thuyết trình 20 phút

Phân bổ 20 phút: mở đầu (1 phút), bài toán (2 phút), giải pháp (2 phút), kiến trúc (3 phút), các luồng + bảo mật (6 phút), công nghệ (3 phút), demo sống (2 phút), kết luận (1 phút), còn lại cho Q&A.

### 3.1. 0:00-1:00 - Mở đầu (slide 1)

"Em xin chào thầy. Nhóm em gồm Nguyễn Lưu Tấn Sang, Vũ Ngọc Sơn và Đỗ Xuân Trí, lớp D22CQCNPM01-N. Đề tài của nhóm em là Xây dựng ứng dụng giao việc và chia sẻ tài liệu số trong tổ chức, với định hướng áp dụng cho môi trường cơ quan nhà nước như Ủy ban nhân dân."

### 3.2. 1:00-3:00 - Bài toán và giải pháp (slide 2-3)

"Trong một UBND, một công việc thường đi qua nhiều phòng ban và nhiều cán bộ, kèm theo hồ sơ, văn bản nhạy cảm. Nếu tài liệu chỉ gửi qua email hay đường dẫn thông thường thì rất khó kiểm soát ai được xem, ai được tải, ai được chia sẻ tiếp, và quyền còn tồn tại hay không sau khi công việc kết thúc. Vì vậy nhóm em xây dựng hệ thống gắn quyền truy cập tài liệu với công việc: người dùng chỉ được cấp đúng quyền cần thiết và quyền có thể tự hết hạn theo deadline của công việc."

### 3.3. 3:00-6:00 - Kiến trúc hệ thống (slide 4-6)

"Kiến trúc của nhóm em gồm một API Gateway và chín service nghiệp vụ. Client Web hoặc Mobile không gọi trực tiếp từng service mà chỉ gọi API Gateway. Gateway xác thực JWT, giới hạn tốc độ và định tuyến request. Mỗi service sở hữu database riêng, không truy cập database của service khác mà giao tiếp qua HTTP đồng bộ hoặc RabbitMQ bất đồng bộ."

Trình bày nhanh vai trò 9 service theo bảng trong slide (auth, user-role, task, document, document-security, permission, audit-log, notification, security-monitoring). Nhấn mạnh: database-per-service; HTTP cho thao tác cần phản hồi ngay (vd Task Service gọi Permission Service để kiểm tra quyền); RabbitMQ cho sự kiện nghiệp vụ bất đồng bộ (vd tạo thông báo sau khi tạo task).

### 3.4. 6:00-9:00 - Luồng nghiệp vụ và mô hình quyền (slide 7-8)

"Một chu kỳ công việc: lãnh đạo tạo task và giao việc, upload tài liệu, gắn tài liệu vào task, cấp quyền PREVIEW và DOWNLOAD. Chuyên viên chỉ thấy đúng những gì được phép. Sau khi xử lý và nộp kết quả, lãnh đạo review và phê duyệt."

Giải thích task-derived grant: mỗi grant bắt buộc có source\_task\_id; thời hạn hiệu lực = thời hạn ngắn nhất giữa grant, deadline task và grant cha; ủy quyền chỉ cấp tập con quyền và không dài hạn hơn quyền gốc.

### 3.5. 9:00-15:00 - Bảo mật (slide 9-16)

**Luồng upload (slide 9):** "File được đưa qua Document Security Service: ghi vùng tạm, quét ClamAV. Nếu có virus thì từ chối upload. Nếu sạch thì mã hóa AES-256-GCM, ciphertext lưu vào MinIO, PostgreSQL chỉ lưu metadata. PostgreSQL không lưu bytes file."

**Luồng tải (slide 10):** "Browser không nhận credential MinIO. Mỗi lần tải phải có download ticket dùng một lần, có thời hạn, gắn với actor và version; hệ thống kiểm tra lại quyền, lấy ciphertext, xác minh HMAC, giải mã và kiểm tra SHA-256 checksum."

**Xác thực (slide 11):** JWT ký số; gateway kiểm tra chữ ký và hạn trước khi route; refresh token rotation; đăng xuất thu hồi session. JWT là ký, không phải mã hóa - không nhét dữ liệu nhạy cảm vào payload.

**Phân quyền (slide 12):** RBAC (ADMIN/EMPLOYEE) + capability. ADMIN quản trị hệ thống nhưng không dùng nội dung; EMPLOYEE mới được giữ grant/tham gia task. Nguyên tắc least privilege.

**Fail-closed (slide 13):** mọi check quyền mặc định từ chối; nếu Permission Service lỗi hoặc timeout thì trả denied chứ không bao giờ tự động allow.

**Mã hóa và chữ ký (slide 14):** AES-256-GCM; DEK mã hóa nội dung từng phiên bản, KEK versioned bảo vệ DEK. Toàn vẹn dùng HMAC + SHA-256. Phân biệt rõ mã hóa (bí mật) và chữ ký (nguồn gốc/toàn vẹn).

**Audit log (slide 15):** nhật ký append-only dạng hash-chain. Mỗi bản ghi có current\_hash tính từ payload và hash bản trước; sửa một bản ghi sẽ phá vỡ chuỗi. Chỉ một writer duy nhất để tránh fork chuỗi.

**Security monitoring (slide 16):** hệ thống ghi nhận sự kiện như đăng nhập thất bại, truy cập bị từ chối. Khi số lần vượt ngưỡng trong cửa sổ thời gian (ví dụ 3 lần/15 phút), hệ thống tạo cảnh báo và có thể thu hồi session theo chính sách.

### 3.6. 15:00-18:00 - Công nghệ và vì sao chọn (slide 17)

"Nhóm em chọn Node.js và NestJS vì TypeScript, module rõ ràng, phù hợp REST API và microservices; NestJS hỗ trợ dependency injection, guard, interceptor, validation. PostgreSQL + Prisma cho dữ liệu quan hệ rõ ràng và type-safe. RabbitMQ cho event bất đồng bộ; Docker Compose đóng gói và triển khai nhất quán."

### 3.7. 18:00-19:00 - Demo sống (slide 18)

Demo 2 tài khoản (lãnh đạo / chuyên viên): tạo task, upload tài liệu, gắn vào task, cấp quyền, đăng nhập chuyên viên xem tài liệu, thử thao tác bị từ chối, nộp kết quả, lãnh đạo phê duyệt, xem audit log và cảnh báo bảo mật ở màn hình Admin.

### 3.8. 19:00-20:00 - Kết luận và Q&A (slide 19-20)

"Điểm chính của hệ thống không chỉ là tạo task hay upload tài liệu mà là kiểm soát toàn bộ vòng đời truy cập: ai được truy cập, được làm gì, trong bao lâu, và hoạt động đó có được ghi nhận hay không. Nhóm em xin cảm ơn thầy và sẵn sàng nhận câu hỏi."

## 4. Lý thuyết công nghệ

### 4.1. Node.js

- **Nền tảng:** chạy JavaScript/TypeScript phía server trên V8 engine của Chrome.
- **Event loop & non-blocking I/O:** một thread chính xử lý hàng nghìn kết nối; thao tác I/O (network, DB, file) không chặn luồng, dùng callback/promise. Phù hợp hệ thống API bị giới hạn bởi I/O hơn là CPU.
- **Hệ sinh thái:** npm/pnpm lớn; cùng ngôn ngữ cho backend và frontend giúp chia sẻ type/contract.
- **Hạn chế:** không lý tưởng cho tác vụ tính toán CPU nặng; cần thiết kế async đúng đắn.

### 4.2. NestJS

- **Framework** cho Node.js, viết bằng TypeScript, có cấu trúc rõ ràng theo module.
- **Dependency Injection (DI):** quản lý provider/service, dễ test và thay thế.
- **Guard, Interceptor, Pipe:** bảo vệ endpoint (xác thực/phân quyền), chuyển đổi dữ liệu, validation.
- **Decorators + module:** tổ chức code theo feature (controller, service, module), tách lớp rõ ràng.
- **Hỗ trợ microservices:** client giao tiếp qua transport (TCP, RabbitMQ, Redis...), phù hợp mô hình đồ án.

### 4.3. Vì sao chọn Node.js/NestJS?

So sánh ngắn với các lựa chọn phổ biến (giải thích nếu thấy hỏi):

| Tiêu chí      | Node.js + NestJS (đã chọn)                     | Spring Boot (Java)               | Django (Python)                   |
|---------------|------------------------------------------------|----------------------------------|-----------------------------------|
| Ngôn ngữ      | TypeScript - tĩnh hóa, an toàn kiểu            | Java - trưởng thành, nặng nề hơn | Python - viết nhanh, kiểm thử rõ  |
| Hiệu năng I/O | Non-blocking, tốt cho API nhiều kết nối        | Thread-per-request, mạnh CPU     | Đủ dùng, kém hơn khi tải cao      |
| Cấu trúc      | Module + DI + guard/interceptor                | Spring MVC/DI quen thuộc         | MVT, admin built-in               |
| Phù hợp đồ án | Microservices nhỏ gọn, một ngôn ngữ xuyên suốt | Quen thuộc nếu team giỏi Java    | Đơn giản, ít hơn cho microservice |

Lựa chọn của nhóm: **TypeScript + NestJS** vì cấu trúc module/DI/guard rõ ràng, phù hợp xây dựng REST API và microservices; TypeScript giúp chia sẻ type và contract giữa các module, tăng độ an toàn khi hệ thống nhiều service.

#### 4.4. PostgreSQL và Prisma

- **PostgreSQL:** CSDL quan hệ mạnh, hỗ trợ transaction, constraint, JSONB; phù hợp dữ liệu nghiệp vụ có quan hệ rõ (task, user, grant, audit event).
- **Prisma ORM:** khai báo schema, sinh client type-safe, migration; truy vấn có kiểm tra kiểu lúc biên dịch.
- **Database-per-service:** mỗi service sở hữu DB riêng, giảm coupling; đổi lại không join trực tiếp được giữa các DB mà phải giao tiếp qua API hoặc event.

#### 4.5. RabbitMQ (so với Kafka)

- **RabbitMQ:** message broker, routing linh hoạt (exchange/binding), phù hợp event nghiệp vụ ở quy mô vừa; publisher không phải chờ consumer xử lý xong.
- **Kafka:** log phân phối, throughput rất lớn, lưu và replay event lâu dài - phù hợp khi cần lưu stream khối lượng lớn.
- Lựa chọn: **RabbitMQ** cho phạm vi đồ án (đơn giản, đủ cho domain events); ghi chú rõ vì sao chưa dùng Kafka.

#### 4.6. Docker và Docker Compose

- **Docker:** đóng gói ứng dụng cùng môi trường chạy thành image; triển khai nhất quán giữa máy dev và server.
- **Docker Compose:** định nghĩa toàn bộ stack (10 service + PostgreSQL, Redis, RabbitMQ, MinIO, ClamAV) trong một file; khởi động, health check, log tập trung.

#### 4.7. Các thành phần hạ tầng khác

| Thành phần | Vai trò                                                          |
|------------|------------------------------------------------------------------|
| Redis      | Lưu session đăng nhập và cache; thu hồi phiên nhanh.             |
| MinIO      | Object storage tương thích S3 - chỉ lưu ciphertext của tài liệu. |
| ClamAV     | Quét virus/malware cho file upload trước khi lưu lâu dài.        |
| RabbitMQ   | Trung chuyển domain events bất đồng bộ giữa các service.         |

Lưu ý giải thích nếu thấy hỏi: trong Docker Compose các container gọi nhau bằng tên service trên Docker network (vd document-security-service dùng host minio, clamav) chứ không dùng localhost - localhost trong container chỉ trỏ về chính container đó.

## 5. Kiến trúc hệ thống và cách vận hành

### 5.1. Tổng quan

Mô hình tổng thể: Web (Next.js) / Mobile (Flutter) - API Gateway - 9 microservice. Mỗi service có database riêng (9 database trên cùng PostgreSQL instance), giao tiếp qua REST (HTTP) và RabbitMQ.

### 5.2. Danh sách service

| Service                   | Trách nhiệm                                              | DB                     | Port |
|---------------------------|----------------------------------------------------------|------------------------|------|
| API Gateway               | JWT validation, rate limiting, định tuyến                | -                      | 3000 |
| Authentication & Identity | Đăng ký, đăng nhập, refresh rotation, logout, session    | auth_db                | 3001 |
| User & Role Management    | User CRUD, lock/unlock, capability grant/revoke          | user_role_db           | 3002 |
| Task Management           | Vòng đời task, comment, submission, review, participants | task_db                | 3003 |
| Document Management       | Metadata, phiên bản, download ticket, records, lưu trữ   | document_db            | 3004 |
| Document Security         | Security pipeline: scan, encrypt, sign, store; KEK       | document_security_db   | 3005 |
| Permission                | Grant CRUD, ủy quyền, cascade revoke, fail-closed check  | permission_db          | 3006 |
| Audit Log                 | Append-only hash-chain audit, verify chain               | audit_db               | 3007 |
| Notification              | Thông báo trong ứng dụng, đọc, tùy chọn                  | notification_db        | 3008 |
| Security Monitoring       | Sự kiện bảo mật, rule/threshold, cảnh báo                | security_monitoring_db | 3009 |



Nguyên tắc: client chỉ gọi Gateway, không gọi port service 3001-3009, không gọi /internal/, không đọc database của service, không nhận secret storage/KEK. audit-log-service chạy đúng một replica (single writer).

### 5.3. Cách giao tiếp giữa các service

- **HTTP đồng bộ:** thao tác cần kết quả ngay - vd Task Service gọi Permission Service để kiểm tra quyền, Document Management gọi Document Security để tạo download ticket.
- **RabbitMQ bất đồng bộ:** domain events (exchange c17.domain) - vd sau khi tạo task, các service khác nhận event để ghi audit, tạo thông báo; audit-log-service và notification-service là consumer.
- **Database-per-service:** không service nào đọc database của service khác.

### 5.4. Hạ tầng và triển khai

- Docker Compose định nghĩa toàn bộ: 10 app + PostgreSQL 16, Redis 7, RabbitMQ 3.13, MinIO, ClamAV.
- CI/CD bằng GitHub Actions: verify (lint, unit test, build, E2E) -> build image backend/web -> deploy lên server EC2 qua SSH.
- Web được build với NEXT\_PUBLIC\_API\_BASE\_URL trỏ về gateway public; mobile dùng chung gateway.

```
pnpm backend:build      # build 10 ứng dụng
pnpm backend:test       # unit tests
pnpm backend:test:e2e   # end-to-end tests
pnpm backend:verify     # full verification
docker compose up -d --build --wait # chạy toàn bộ stack
```

## 6. Các luồng hệ thống chính

### 6.1. Luồng xác thực (login / refresh / logout)

```
Client -> API Gateway -> Authentication & Identity Service -> auth_db
1) POST /auth/login (email + password)
   - xác thực, kiểm tra tài khoản bị khóa
   - cấp access token (JWT) + refresh token; lưu session (Redis)
2) GET /workspace/... (mang JWT)
   - Gateway xác minh chữ ký + hạn của JWT rồi mới route
3) POST /auth/refresh -> rotate token: cấp token mới, vô hiệu token cũ
4) POST /auth/logout -> thu hồi refresh token, xóa session
```

### 6.2. Luồng tạo task và phân công

```
1) Người giao (creator) gọi POST /tasks
2) Task Management Service ghi task (creator, assignee, deadline, trạng thái)
3) Phát domain event task.created (RabbitMQ)
   - Notification: tạo thông báo cho assignee
   - Audit Log: ghi AuditEvent task.created
```

### 6.3. Luồng upload tài liệu (security pipeline)

- 1) Client gửi multipart upload -> Gateway -> Document Management Service
- 2) Document Security Service nhận file:
  - ghi file vào vùng tạm (plaintext tạm)
  - quét ClamAV qua TCP :3310 (INSTREAM)
  - FOUND -> từ chối upload, dọn file tạm
  - OK -> AES-256-GCM: sinh DEK, KEK bảo vệ DEK
  - lưu ciphertext vào MinIO (:9000)
  - lưu metadata (object key, checksum, encrypted DEK, IV, tag, KEK version, size)
- 3) dọn plaintext/ciphertext tạm

### 6.4. Luồng cấp quyền (grant) gắn với task

- 1) Người giao việc chọn tài liệu trong task -> POST /grants  
body: { actor\_id, resource\_id, source\_task\_id, actions, expires\_at }
- 2) Permission Service kiểm tra:
  - actor là EMPLOYEE (ADMIN không được nhận quyền nội dung)
  - grant phải có source\_task\_id hợp lệ
  - SHARE phải đi kèm PREVIEW hoặc DOWNLOAD
  - thời hạn <= deadline của task
- 3) Ghi Grant; phát event DOCUMENT\_GRANT\_CREATED\_IN\_TASK

### 6.5. Luồng xem / tải tài liệu

- 1) Client xin quyền: Task/Document Service gọi permission-service check
  - kiểm tra grant hợp lệ, chưa hết hạn (effective\_expires\_at)
  - lỗi/timeout -> deny (fail-closed)
- 2) Xem trước: tạo preview session (audit DOCUMENT\_PREVIEW\_SESSION\_CREATED)
- 3) Tải xuống: tạo download ticket (1 lần, có hạn, gắn actor+doc+version)
  - Security Service: lấy ciphertext -> verify HMAC -> giải mã -> SHA-256 -> trả file

### 6.6. Luồng xử lý, nộp kết quả và phê duyệt

- 1) Chuyên viên đổi trạng thái task sang IN\_PROGRESS
- 2) Nộp kết quả (submission) -> task chuyển WAITING\_REVIEW
- 3) Lãnh đạo review -> APPROVED (hoặc trả lại)
- 4) Phát event -> notification + audit

### 6.7. Luồng hết hạn / thu hồi quyền

- Thời hạn hiệu lực của grant = min(expires\_at grant, deadline task, expires\_at grant cha) - kiểm tra tại thời điểm request.
- Thu hồi grant cha -> thu hồi luôn grant con (cascade revocation).
- Gỡ tài liệu khỏi task (detach) chỉ được thực hiện bởi người có quyền DISPOSE hoặc chủ tài liệu; grant liên quan bị thu hồi theo task.
- Hệ thống không có quyền tồn tại vô thời hạn; hết việc là hết quyền.

### 6.8. Luồng audit và cảnh báo bảo mật

- 1) Sự kiện quan trọng (tạo task, cấp grant, download, detach bị từ chối...)  
-> Audit Log Service ghi AuditEvent dạng hash-chain
- 2) Sự kiện bảo mật (đăng nhập thất bại, permission denied)  
-> Security Monitoring tăng counter cho (rule, actor, window)
  - counter >= threshold (vd 3 lần/15 phút) -> tạo SecurityAlert (OPEN)
  - chính sách ALERT hoặc BLOCK; có thể thu hồi session

## 7. Bảo mật hệ thống

### 7.1. Mô hình phân quyền

- **RBAC:** hai vai trò hệ thống ADMIN và EMPLOYEE.
- **Capability:** quyền chi tiết hơn role, vd ARCHIVE\_RECEIVE (người lưu trữ). Capability có tính chất gắn nội dung không được cấp cho ADMIN.
- **Task-derived grant:** quyền tài liệu phải gắn với task; thời hạn bị khóa theo task.

### 7.2. Các nguyên tắc bảo mật

| Nguyên tắc                   | Ý nghĩa                                                                      |
|------------------------------|------------------------------------------------------------------------------|
| Default deny                 | Mọi check quyền trả denied nếu không có grant hợp lệ.                        |
| Fail closed                  | Permission Service lỗi/timeout -> denied, không bao giờ allow mặc định.      |
| Least privilege              | Chỉ cấp quyền tối thiểu cần thiết cho công việc.                             |
| ADMIN không đụng nội dung    | Quản trị hệ thống, không xem/chia sẻ nội dung tài liệu, không tham gia task. |
| EMPLOYEE là vai trò nội dung | Chỉ EMPLOYEE được giữ grant hoặc tham gia task.                              |
| Kiểm tra tại thời điểm dùng  | Expiry luôn được kiểm tra khi request, không tin trạng thái cũ.              |
| Comment là bí mật            | Nội dung comment không ghi vào audit log.                                    |

### 7.3. Xác thực và phiên

- JWT ký số (HS256/RS256 tùy cấu hình), có hạn; gateway kiểm tra trước khi route.
- Refresh token rotation: mỗi lần refresh cấp token mới và vô hiệu token cũ.
- Session lưu Redis, hỗ trợ đăng xuất và thu hồi phiên từ xa.
- Rate limiting trên gateway chống spam/brute-force.

### 7.4. Mã hóa và toàn vẹn nội dung

- AES-256-GCM: bảo mật nội dung; mỗi phiên bản có DEK riêng; KEK versioned bảo vệ DEK (ADR-0003).
- HMAC integrity signature + IV + authentication tag + SHA-256 checksum khi tải xuống.
- MinIO chỉ chứa ciphertext; PostgreSQL chỉ lưu metadata; client không nhận object key/credential.
- ClamAV chặn upload file chứa virus trước khi lưu lâu dài.

### 7.5. Một số kịch bản tấn công đã được thiết kế chống

| Kịch bản                        | Cách hệ thống chống                                          |
|---------------------------------|--------------------------------------------------------------|
| Token bị lộ                     | Token có hạn ngắn; refresh rotation; thu hồi session.        |
| Permission service bị lỗi       | Fail-closed trả denied.                                      |
| ADMIN tự cấp quyền đọc nội dung | ADMIN không thể tham gia task/giữ grant nội dung (ADR-0004). |
| Mention dùng để mở quyền        | Mention không tạo quyền; chỉ mention người đã có quyền.      |
| Upload file độc hại             | ClamAV scan trước khi lưu.                                   |
| Sửa/xóa nhật ký                 | Audit append-only + hash-chain phát hiện sai lệch.           |
| Truy cập sau khi hết hạn        | Expiry kiểm tra tại request; download qua ticket.            |

## 8. Ghi log nhật ký (Audit Log)

### 8.1. Khái niệm

Audit log là nhật ký lưu các sự kiện quan trọng của hệ thống phục vụ đối chiếu, điều tra và tuân thủ. Thiết kế là **append-only**: chỉ được ghi thêm, không sửa/xóa tùy ý. Nội dung tài liệu và comment **không bao giờ** được đưa vào audit event (tránh rò rỉ dữ liệu nhạy cảm qua log).

## 8.2. Cấu trúc hash-chain (ADR-0002)

```
current_hash = SHA-256(canonical_payload + previous_hash)
```

```
AuditEvent { id, event_type, actor_id, resource_id, payload,  
              prev_hash, current_hash, created_at }
```

Mỗi bản ghi mới bám vào hash bản trước. Sửa/xóa một bản ghi sẽ làm vỡ chuỗi -> khi verify toàn bộ chuỗi phát hiện vị trí sai lệch.

## 8.3. Single writer và chống trùng

- audit-log-service chạy đúng một replica; consume với prefetch=1.
- Mỗi append nằm trong một transaction có lock trên chain-head.
- Unique index trên event\_id + dedupe trong cùng transaction -> RabbitMQ redelivery không làm fork chuỗi hoặc tạo gap.
- Hệ quả: ingest audit không scale ngang - là giới hạn chủ động vì tamper-evidence quan trọng hơn throughput.

## 8.4. Các loại sự kiện được ghi (ví dụ)

```
task.created          document.created  
TASK_DOCUMENT_ATTACHED  DOCUMENT_GRANT_CREATED_IN_TASK  
DOCUMENT_GRANT_UPDATED_IN_TASK  
DOCUMENT_PREVIEW_SESSION_CREATED  
DOCUMENT_DOWNLOAD_REDEEMED  
TASK_DOCUMENT_DETACH_DENIED / TASK_DOCUMENT_DETACHED  
DOCUMENT_GRANTS_REVOKED_DUE_TO_TASK_DETACH  
security.alert.created
```

## 8.5. Kiểm chứng chuỗi

Verify là duyệt tuyến tính từ bản ghi gốc (genesis) đến cuối: tính lại hash từng bản ghi và so sánh; bất kỳ khác biệt nào cho biết dữ liệu đã bị can thiệp. Đã có bài test phát hiện giả mạo (tamper detection).

## 8.6. Log vận hành

- Ngoài audit log, các service ghi log vận hành (request/response, lỗi, health check) qua cơ chế log chuẩn của NestJS; docker compose cho phép xem log tập trung (docker compose logs).
- Nguyên tắc: không ghi password, token, secret, object key, KEK vào log.

## 9. Kịch bản demo thao tác (từng bước trên UI)

### 9.1. Chuẩn bị trước

- Mở web (URL server public) và gateway; chuẩn bị 2 tài khoản: lãnh đạo + chuyên viên (có thể dùng cửa sổ ẩn danh).
- Chuẩn bị sẵn 1 file mẫu (PDF/DOCX) và 1 task mẫu để phòng upload không ổn định.
- Không trình bày password/JWT/secret/database trên màn hình.

### 9.2. Các bước demo

7. Đăng nhập tài khoản **lãnh đạo**. Nói: "Sau khi đăng nhập hệ thống cấp JWT + session; các request đi qua Gateway."
8. Vào **Tasks > Create Task**: tiêu đề "Xử lý hồ sơ đề xuất dự án chuyển đổi số", giao cho chuyên viên, đặt deadline. Nói về vòng đời trạng thái task.
9. Vào **Documents**: upload file mẫu. Nói ngắn về security pipeline (scan, encrypt, MinIO).
10. Mở task > **Attach Document** để gắn tài liệu. Nói: "Quan hệ task-document là căn cứ để cấp quyền."
11. Vào **Grants**: cấp quyền **PREVIEW, DOWNLOAD** cho chuyên viên, có thời hạn. Nói: không cấp UPDATE/SHARE; grant có hạn.
12. Đăng nhập tài khoản **chuyên viên** (cửa sổ ẩn danh). Nói: "Chuyên viên chỉ thấy đúng task và tài liệu được phép."
13. Mở tài liệu, thử tải (nếu được). Nói về download ticket. Thử một thao tác không được phép -> server trả denied.
14. Chuyên viên đổi task sang IN\_PROGRESS, nhập kết quả, nộp -> task sang chờ review.
15. Quay lại tài khoản lãnh đạo, mở task, review -> **APPROVED**. Nói quyền hết hạn theo deadline.
16. Đăng nhập **Admin**: xem người dùng/capability, xem **audit log** và **cảnh báo bảo mật** (nếu có).

### 9.3. Nếu gặp sự cố khi demo

- Upload lỗi -> dùng tài liệu mẫu đã chuẩn bị hoặc minh họa bằng Postman (docs/c17-api-postman-collection.json).
- Server lỗi -> chuyển sang trình bày kiến trúc, sequence flow và các slide bảo mật.
- Chỉ khẳng định phần đã kiểm tra thực tế; phần mới ở mức mã nguồn/thiết kế thì nói rõ mức đó.

## 10. Câu hỏi dự kiến và cách trả lời

### 10.1. Vì sao không làm Monolith?

"Monolith đơn giản hơn khi bắt đầu, nhưng đề tài yêu cầu nghiên cứu microservices. Nhóm em tách service theo domain để mỗi service có trách nhiệm rõ, database riêng, triển khai độc lập. Trade-off là microservices phức tạp hơn ở giao tiếp, giám sát và nhất quán dữ liệu."

### 10.2. Nếu Permission Service bị lỗi thì sao?

"Hệ thống fail-closed: timeout hoặc lỗi thì trả denied, không mở tài liệu mặc định."

### 10.3. Người được giao có thể cấp quyền vô hạn không?

"Không. Grant con phải là tập con quyền của grant cha và không thể có thời hạn dài hơn grant cha."

### 10.4. JWT có phải mã hóa không?

"JWT thường được ký, không đồng nghĩa với mã hóa. Chữ ký giúp xác minh token không bị sửa; vì vậy không đưa dữ liệu nhạy cảm vào payload JWT."

### 10.5. Admin có xem được tài liệu không?

"Không theo chính sách nghiệp vụ. Admin quản lý tài khoản, role, capability và monitoring; không mặc định có quyền đọc hoặc chia sẻ nội dung tài liệu."

### 10.6. Audit Log có thể bị sửa không?

"Thiết kế audit là append-only và hash-chain. Mỗi event liên kết với hash của event trước; khi kiểm tra lại chuỗi, hệ thống phát hiện sai lệch."

### 10.7. Có đảm bảo đúng quy định pháp luật không?

"Phần mềm hỗ trợ các nguyên tắc như phân quyền, bảo vệ dữ liệu, ghi vết, phát hiện truy cập trái phép. Việc tuân thủ đầy đủ còn phụ thuộc vào phân loại tài liệu, quy trình nghiệp vụ, hạ tầng, chứng thư số và quy chế cụ thể của từng cơ quan."

## 10.8. Đồ án đã hoàn thiện hết chưa?

"Nhóm đã xây dựng kiến trúc microservices, các service backend chính, API Gateway, database schema, permission flow, audit flow và các màn hình Web chính. Phần nhóm tiếp tục xác minh là runtime end-to-end trên server, đặc biệt là toàn bộ pipeline upload file, kết nối object storage, malware scan và một số luồng event bất đồng bộ. Nhóm phân biệt rõ ba mức: có mã nguồn, có kiểm thử ở mức service, và đã xác minh chạy thực tế."

## 10.9. Mobile đã hoàn thiện chưa?

"Web là bề mặt demo chính. Mobile được thiết kế dùng chung API Gateway và contract backend; nhóm không tuyên bố mobile hoàn thiện nếu chưa có bản chạy thực tế."

# 11. Kết luận và hướng phát triển

## 11.1. Kết luận

Điểm chính của hệ thống không chỉ là tạo task hay upload tài liệu, mà là **kiểm soát toàn bộ vòng đời truy cập**: ai được truy cập, được làm gì, trong bao lâu, và hoạt động đó có được ghi nhận hay không. Kiến trúc microservices kết hợp các nguyên tắc bảo mật (default deny, fail closed, least privilege, tách biệt quản trị - nội dung, mã hóa, hash-chain audit) giúp hệ thống phù hợp với bài toán chia sẻ tài liệu nhạy cảm trong tổ chức như UBND.

## 11.2. Hạn chế hiện tại

- Một số luồng runtime end-to-end trên server chưa được xác minh đầy đủ (pipeline file, object storage, malware scan).
- Mobile client mới ở mức thiết kế.
- Microservices phức tạp hơn ở giám sát, log tập trung và nhất quán dữ liệu.

## 11.3. Hướng phát triển

- Hoàn thiện kiểm thử E2E toàn bộ luồng và xác minh runtime trên server.
- Mở rộng object storage sang AWS S3 / Cloudflare R2 qua cấu hình.
- Giám sát tập trung (metrics, tracing, alerting).
- Tích hợp chữ ký số cơ quan, OTP/bảo mật hai lớp cho tài liệu mật.
- Hoàn thiện Flutter mobile dùng chung gateway.

# Phụ lục A. Từ vựng miền chính



|                  |                                                                                           |
|------------------|-------------------------------------------------------------------------------------------|
| ADMIN            | Vai trò quản trị hệ thống: người dùng, role, capability, cảnh báo. Không đựng nội dung.   |
| EMPLOYEE         | Vai trò thực hiện công việc; vai trò duy nhất được giữ grant/tham gia task.               |
| Task             | Đơn vị công việc với deadline, vòng đời, creator, assignee; là căn cứ cấp quyền tài liệu. |
| Grant            | Quyền tài liệu gắn với task, có thời hạn; bắt buộc source_task_id.                        |
| Capability       | Quyền chi tiết hơn role, do ADMIN cấp (vd ARCHIVE_RECEIVE).                               |
| Participant      | Người được phép trên một task cụ thể (creator, assignee, participant được gán).           |
| Download ticket  | Chứng chỉ tải tài liệu dùng một lần, có hạn, gắn actor+version.                           |
| Effective expiry | Thời hạn hiệu lực thực tế = min(grant, task deadline, grant cha).                         |
| Hash-chain audit | Chuỗi bản ghi mà mỗi bản ghi chứa hash của bản trước; chống giả mạo.                      |
| Fail closed      | Khi lỗi/thời gian chờ -> từ chối thay vì cho phép mặc định.                               |

## Phụ lục B. Checklist trước giờ demo

- Kiểm tra URL server và web; đăng nhập thử cả 2 tài khoản.
- Chuẩn bị sẵn task mẫu, tài liệu mẫu và grant PREVIEW/DOWNLOAD.
- Mở sẵn 2 trình duyệt / cửa sổ ẩn danh.
- Ghi lại task ID/user ID nếu cần dùng Postman.
- Không trình bày password, JWT, secret, object key, database.
- Kiểm tra file slide HTML và tài liệu DOCX này trước khi thuyết trình.

## Phụ lục C. Tài liệu liên quan trong repo

- docs/presentations/C17-slides-20min.html - slide thuyết trình (file này).
- docs/presentations/C17-thuyet-trinh-20min.docx - tài liệu này.
- docs/presentations/C17-demo-presentation-script.md - script demo 10-15 phút bản trước.
- README.md, docs/adr/ - kiến trúc và quyết định thiết kế.
- docs/c17-api-postman-collection.json - bộ test API bằng Postman.